Generated V2 pack with architecture details and screenshot appendix.

OPTransformer Deep Development Handbook (V3)

Date: 2026-10-19

Purpose: Provide a reconstruction-grade handbook for building an application equivalent to OPTransformer from backend to frontend, in controlled phases, with detailed contracts, mapping logic, transformation language semantics, dependencies, toggles, and testing guardrails.

This handbook is generated from the active `docs/` corpus and runtime manifests in the main project directory.

---

0) How to use this handbook

- Treat this as the implementation source for Claude/Copilot-style phase execution.
- Keep all changes additive and backward-compatible unless a phase explicitly says otherwise.
- Always preserve invariants listed in Section 2.
- Use screenshot appendix (end of PDF) to map conceptual sections to UI behavior.

---

1) System overview

1.1 Product objective

OPTransformer is a schema-driven transformation platform for financial messaging workflows. It ingests schemas (XSD, JSON schema, MT/ACH/fixed-length variants), builds mappings, applies transformation rules, validates outputs, and supports flow orchestration and export/test-run pathways.

1.2 Runtime topology

- Backend: Spring Boot + Java 17 + MongoDB
- Frontend: React CRA app (`xsd-ui`)
- Data store: MongoDB (`optransformer`)

1.3 Major capabilities

1. Schema upload/parse/validation
2. Mapping authoring and normalization
3. Function-driven transformation rules
4. Mapping intelligence and data-lake style reuse
5. Flow authoring/execution/simulation
6. Message class reusable definitions
7. MT-to-MX preview generation (feature-flagged)
8. Auth/admin/access controls
9. AI chat assistant endpoints

---

2) Non-negotiable invariants and guardrails

1. Preserve latest schema retrieval by `(messageType, version)` with latest-by-id semantics.
2. Parse flows must persist entities before returning responses.
3. Java and frontend normalizers must remain behaviorally aligned.
4. Frontend API calls must route through centralized API helpers.
5. Preserve CORS + credential behavior unless explicitly requested.
6. Keep upload limits/toggles stable by default.
7. Preview-only modules (especially MT-to-MX) must not trigger submit/release side effects.

---

3) Repository and docs corpus map

3.1 Active engineering docs in `docs/`

- `TDD.md` (architecture + endpoint inventory + pipeline details)
- `TECHNICAL_DOCUMENT.md` (current implementation snapshot)
- `AUTH_MODULE_SPEC.md` (JWT/refresh/CSRF/RBAC behavior)
- `TRANSFORMATION_LANGUAGE.md` (rule language semantics)
- `ENGINE_API.md` (typed transform engine facade)
- `MDB_MIGRATION_ROLLBACK.md` (MDB rollout/rollback toggles)
- `MDB_JAR_EXPORT_SPEC.md` (headless test-run + export-jar contracts)
- `MESSAGE_CLASS_MODULE.md` (message class domain and APIs)
- `OPTRANSFORMER_JSON_SCHEMA_MULTI_CHILD_COMPOSITION_FIXES.md` (composition constraints/DoD)
- `06_TEST_ACCEPTANCE_CRITERIA.md` (semantic acceptance checks)
- `PHASE_*.md` and flow phase docs under `docs/flow/` (incremental rollout details)
- `02_SAMPLE_SNIPPETS_AND_EXPECTED_OUTPUTS.md`, `03_PATTERN_INVENTORY.md` (rule examples and pattern taxonomy)

3.2 Prompt/spec artifacts in docs

Several files are implementation prompts and design guidance. Use them as constraints and test criteria, not as guaranteed current behavior, unless confirmed in code.

---

4) Backend architecture and modules

4.1 Layered architecture

- Controllers expose HTTP contracts.
- Services implement parsing, normalization, mapping, transformations, orchestration.
- Mongo repositories persist document models.
- Security/auth module enforces authz/authn and module-level access.

4.2 Controller domains (high-level)

- Schema core (`/schema/*`)
- Mapping intelligence (`/mapping-intelligence/*`)
- MDB (`/mdb/*`)
- Functions (`/functions/*`)
- Flow APIs (`/flow/*`, `/flow-orchestration/*`, `/api/flows/*`)
- Message classes (`/api/projects/{projectId}/message-classes/*`)
- Product owner (`/po/*`)
- Sample mapping jobs (`/sample-mapping/*`)
- Cartridge intelligence/import (`/cartridge-intelligence/*`, `/api/mapping-builder/import/cartridge-csv/*`)
- MT-to-MX (`/mt-to-mx/*`)
- AI chat (`/api/ai/*`)
- Auth/Admin/Access (`/auth/*`, `/admin/*`, `/access/*`)

---

5) Frontend architecture and UI composition

5.1 SPA shell model

- Panel-driven app shell (state-based screen switching)
- Centralized API helper usage and auth-aware requests (`credentials: include` behavior)
- Mapping-focused utilities and studio runtime

5.2 Core UI modules

- `SchemaWorkbench`
- `MappingDocumentBuilder`
- `MappingIntelligence`
- `CartridgeIntelligence`
- `FlowBuilder`, `FlowOrchestrator`
- `FunctionLibrary`
- `MessageClassStudio`
- `MtPackageStudio`, `MtToMxPreview`
- `ProductOwnerPortal`, `AdminDashboard`, `AiChatAssistant`

5.3 Studio/runtime files to mirror

9.2 Function APIs

- `GET /functions?category=&search=`
- `GET /functions/categories`
- `GET /functions/{id}`
- `POST /functions/{id}/invoke`
- `POST /functions/evaluate`

Invoke example:

```json
{
  "args": ["2026-05-20", "dd/MM/yyyy"]
}
```

Result:

```json
{
  "ok": true,
  "result": "20/05/2026"
}
```

9.3 MDB test-run/export

- `POST /mdb/{id}/test-run` -> `{ output, exception }`
- `POST /mdb/{id}/export-jar` -> binary JAR

9.4 Flow execution

Execute request:

```json
{
  "correlationId": "corr-001",
  "input": "<Document>...</Document>",
  "inputFormat": "XML",
  "parameters": {
    "country": "UK"
  },
  "traceLevel": "DEBUG"
}
```

Execute response shape:

```json
{
  "success": true,
  "executionId": "exec-123",
  "correlationId": "corr-001",
  "flowName": "Payment Flow",
  "flowVersion": "1.0",
  "outputFormat": "JSON",
  "output": "{\"status\":\"OK\"}",
  "selectedRoutes": ["default"],
  "invokedMappings": ["PACS008_TO_HEADER"],
  "warnings": [],
  "durationMs": 92,
  "error": null
}
```

---

10) Mapping normalization and intelligence logic

7.2 Frontend dependencies (`xsd-ui/package.json`)

- `react`
- `react-dom`
- `react-scripts`
- `xlsx`
- dev: `madge`

---

8) Data model blueprint and sample documents

8.1 Core collections

- `schemas`
- `mappings`
- `mapping_datalake`
- `flows`
- `mdb_states`
- `mapping_sample_jobs`
- auth collections (`users`, `refresh_tokens`, `password_reset_requests`, etc.)

8.2 Schema document example

```json
{
  "_id": "schemaId",
  "messageType": "pacs.008",
  "version": "001.08",
  "sourceType": "XSD",
  "namespace": "urn:iso:std:iso:20022:tech:xsd:pacs.008.001.08",
  "fields": [
    {
      "name": "Document",
      "path": "/Document",
      "type": "complex",
      "required": true,
      "children": []
    }
  ],
  "xsdContent": "<xsd:schema>...</xsd:schema>"
}
```

8.3 Mapping document example

```json
{
  "_id": "mappingId",
  "name": "PACS008_TO_HEADER",
  "sourceMessageType": "pacs.008",
  "targetMessageType": "internal.header",
  "rules": [
    {
      "sourcePath": "/Document/FIToFICstmrCdtTrf/GrpHdr/MsgId",
      "targetPath": "/header/messageId",
      "expression": "trim(/Document/FIToFICstmrCdtTrf/GrpHdr/MsgId)",
      "transform": "upper"
    }
  ],
  "version": 1,
  "createdAt": "2026-10-19T10:00:00Z"
}
```

8.4 Flow document example

```json
{
```

- `xsd-ui/src/studio/mappingEngine.js`
- `xsd-ui/src/studio/mdbFunctions.js`
- `xsd-ui/src/utils/mappingNormalizer.js`

---

6) Configuration and runtime feature toggles

6.1 Runtime properties (critical)

From `src/main/resources/application.properties`:

- `server.port=8081`
- `spring.data.mongodb.uri=mongodb://localhost:27017/optransformer`
- upload limits: `50MB`
- `mt.to.mx.generator.enabled=true`
- `mdb.save.new-model.enabled=true`
- `mdb.audit.enabled=true`
- `mdb.project.page-size.default=20`
- `mdb.project.page-size.max=200`
- `engine.v2.enabled=true`
- `message.class.enabled=true`

6.2 Practical toggle strategy

- Keep all toggles externally mutable (properties/env) to enable rollback without redeploy.
- For each migration phase, document default + rollback value.

---

7) Dependency inventory required for reconstruction

7.1 Backend dependencies (`pom.xml`)

Framework/runtime:
- `spring-boot-starter`
- `spring-boot-starter-web`
- `spring-boot-starter-security`
- `spring-boot-starter-validation`
- `spring-boot-starter-data-mongodb`
- `spring-boot-starter-json`

Data serialization:
- `jackson-core`
- `jackson-databind`
- `jackson-annotations`
- `jackson-datatype-jsr310`

Security/auth:
- `io.jsonwebtoken:jjwt-api`
- `io.jsonwebtoken:jjwt-impl` (runtime)
- `io.jsonwebtoken:jjwt-jackson` (runtime)

Parser/processing:
- `org.antlr:antlr4-runtime`
- `org.apache.poi:poi-ooxml` (Excel parsing)
- `org.jsoup:jsoup`

MDB runtime:
- `com.citi:mdb-runtime:1.0.0`

Test dependencies:
- `spring-boot-starter-test`
- `spring-security-test`
- `spring-boot-starter-webmvc-test`

Build plugins:
- `spring-boot-maven-plugin`
- `antlr4-maven-plugin`

```
  "_id": "flow-1",
  "name": "Payment Flow",
  "version": "1.0",
  "status": "DRAFT",
  "nodes": [
    {
      "nodeId": "n1",
      "nodeType": "PARSE",
      "name": "Parse input"
    }
  ],
  "edges": [
    {
      "edgeId": "e1",
      "edgeType": "DEFAULT",
      "fromNodeId": "n1",
      "toNodeId": "n2"
    }
  ]
}
```

---

9) Endpoint contracts and example payloads

9.1 Schema parse/validate

Parse request:

```json
{
  "messageType": "pacs.008",
  "version": "001.08",
  "xmlMessage": "<Document>...</Document>"
}
```

Parse response:

```json
{
  "messageType": "pacs.008",
  "version": "001.08",
  "rootElement": "Document",
  "schemaFound": true,
  "extractedFields": {
    "/Document/FIToFICstmrCdtTrf/GrpHdr/MsgId": ["ABC123"]
  },
  "missingRequiredFields": []
}
```

Validate response:

```json
{
  "valid": false,
  "messageType": "pacs.008",
  "version": "001.08",
  "schemaUnavailable": false,
  "issues": [
    {
      "path": "/Document/FIToFICstmrCdtTrf/GrpHdr/MsgId",
      "severity": "ERROR",
      "message": "Element is required"
    }
  ]
}
```

- Extract source by path
- Apply function-chain or expression
- Apply defaults/coalesce/conditions
- Validate target type/shape
- Emit row-level diagnostics without crashing full run when possible

---

12) Function library: categories and representative signatures

12.1 Core

- `set(value)`

12.2 Date and Time

- `currentDate([format])`
- `currentDateTime([format])`
- `currentTimestamp()`
- `dayOfYear([date], [pattern])`
- `monthOfYear([date], [pattern])`
- `yearOf([date], [pattern])`
- `dayOfMonth([date], [pattern])`
- `dayOfWeek([date], [pattern])`
- `weekOfYear([date], [pattern])`
- `parseDate(text, inputPattern, [outputPattern])`
- `formatDate(isoDate, pattern)`
- `addDays(date, days)`

12.3 String

- `concat(s1, s2, ...)`
- `join(separator, s1, s2, ...)`
- `substring(text, start, [end])`
- `charAt(text, index)`
- `length(text)`
- `upper(text)`, `lower(text)`, `trim(text)`
- `replace(text, search, replacement)`
- `replaceRegex(text, regex, replacement)`
- `padLeft(text, length, [pad])`, `padRight(text, length, [pad])`
- `split(text, delimiter, index)`
- `indexOf(text, search)`, `contains(text, search)`
- `startsWith(text, prefix)`, `endsWith(text, suffix)`
- `reverse(text)`

12.4 Arithmetic

- `add`, `subtract`, `multiply`, `divide`, `modulo`
- `absolute`, `round`, `ceil`, `floor`
- `min`, `max`, `power`

12.5 Conditional

- `ifEmpty(value, fallback)`
- `ifEquals(a, b, whenTrue, whenFalse)`
- `coalesce(v1, v2, ...)`
- `switchCase(input, case1, out1, ..., [default])`
- `ifGreater(a, b, whenTrue, whenFalse)`

12.6 Validation

- `isIBAN(text)`
- `isBIC(text)`
- `isEmail(text)`
- `isNumeric(text)`
- `isDate(text, [pattern])`
- `matches(text, regex)`

12.7 Conversion

10.1 Canonical normalizer pipeline

1. Sheet classification
2. Header detection (rows 1-12)
3. Column role inference
4. Row normalization
5. Transform type inference
6. Confidence scoring
7. Country overlays
8. Ambiguity flags
9. Statistics
10. Template cataloging

10.2 Mapping transform type categories

- `DIRECT`
- `CONDITIONAL`
- `COALESCE`
- `CONSTANT`
- lookup and complex variants

10.3 Confidence and ambiguity handling

- Confidence metrics are stored with normalized rows.
- Ambiguity is preserved explicitly for analyst review.
- Suggest/compare APIs support reuse and conflict analysis.

---

11) Transformation language and rule semantics

11.1 Language characteristics

- Backward-compatible with legacy path expressions.
- Supports `def` variable declarations.
- Statement separators: `;` or newline.
- Path aliases support slash and dot traversal.
- Existence checks: `field.exists`, `section.exists`, `source.exists`.
- Stable diagnostic error code model (`TRN-*`).

11.2 Example rules

Variable and concatenation:

```text
def debtor = /Document/CstmrCdtTrfInitn/PmtInf/Dbtr;
def address = debtor.PstlAdr;
concat(address.TwnNm, ", ", address.Ctry)
```

Conditional fallback:

```text
ifEmpty(/Document/.../Cdtr/Nm, "UNKNOWN_BENEFICIARY")
```

Date conversion:

```text
formatDate(parseDate(/Document/.../CreDtTm, "yyyy-MM-dd'T'HH:mm:ss"), "yyyyMMdd")
```

Section-level pattern (pseudo):

```text
for each Tx in /Document/.../CdtTrfTxInf
  SecAddElement("transactions", mapTx(Tx))
```

11.3 Runtime behavior

- `anyToText(value)`
- `textToAny(value)`
- `createAny(name, value, [namespace])`
- `toBoolean(text)`
- `toNumber(text)`
- `uuid()`
- `epochToDate(epoch, [pattern])`

12.8 Legacy transform pipeline support

Legacy operators still supported in service transform paths:

- `upper`
- `lower`
- `trim`
- `substring:start,end`
- `default:value`
- `padLeft:length,padChar`
- `padRight:length,padChar`

---

13) JSON schema composition logic (multi-child)

13.1 Required behavior

- Parent wrapper path anchors message shape.
- Exactly one selected child schema for a composed document path.
- Resolve child schema deterministically; fail explicitly on ambiguity.
- Do not emit synthetic runtime placeholders such as `[items]` keys.
- Keep parent/child `$ref` scopes isolated.

13.2 Example canonical shape

```json
{
  "envelope": {
    "originalPayment": {
      "document": {
        "grpHdr": {
          "msgId": "MSG-001"
        }
      }
    },
    "enrichedPayment": {
      "document": {
        "audit": {
          "source": "SYSTEM_A"
        }
      }
    }
  }
}
```

13.3 Failure mode examples

- `AMBIGUOUS_CHILD_SCHEMA`
- unresolved child reference
- invalid selected child for target path

---

14) Flow model and orchestration

14.1 Coexistence strategy

- Keep legacy `/flow/*` contracts intact.
- Add native flow model under `/api/flows/*` without breaking old flows.

14.2 Runtime object model

- Node/edge graph
- Validation stage before publish/execute
- Simulate mode for dry-runs
- Execute mode with trace and result diagnostics
- Mapping invocation adapter for compatibility with existing mapping services

14.3 Execution result requirements

- `success` flag
- unique execution identifiers
- selected routes
- invoked mappings
- warnings/errors
- duration metrics

---

15) Message Class module details

15.1 Scope

Reusable message definition entities to reduce duplication and improve governance.

15.2 APIs

- Create/list/get/update/delete
- Validate/publish/version/dependencies
- Test parse / test serialize
- Preview load (JSON + file)
- Capabilities endpoint

15.3 Security and safety

- Project-scoped access checks
- Reject XML DTD/ENTITY attack vectors
- No bytecode execution in class import flows
- Mask sensitive payload traces in audits

---

16) Auth, token, and RBAC details

16.1 Identity policy

- Username format policy enforced (letter+digits style pattern)
- Role model includes ADMIN/BA/DEVELOPER style responsibilities

16.2 Token strategy

- JWT access tokens
- Refresh token rotation and revocation
- Refresh token hashes persisted at rest
- CSRF (double-submit style) on cookie-sensitive paths

16.3 Lockouts and resets

- Configurable failed-attempt thresholds
- Lockout windows
- Password reset TTL workflows

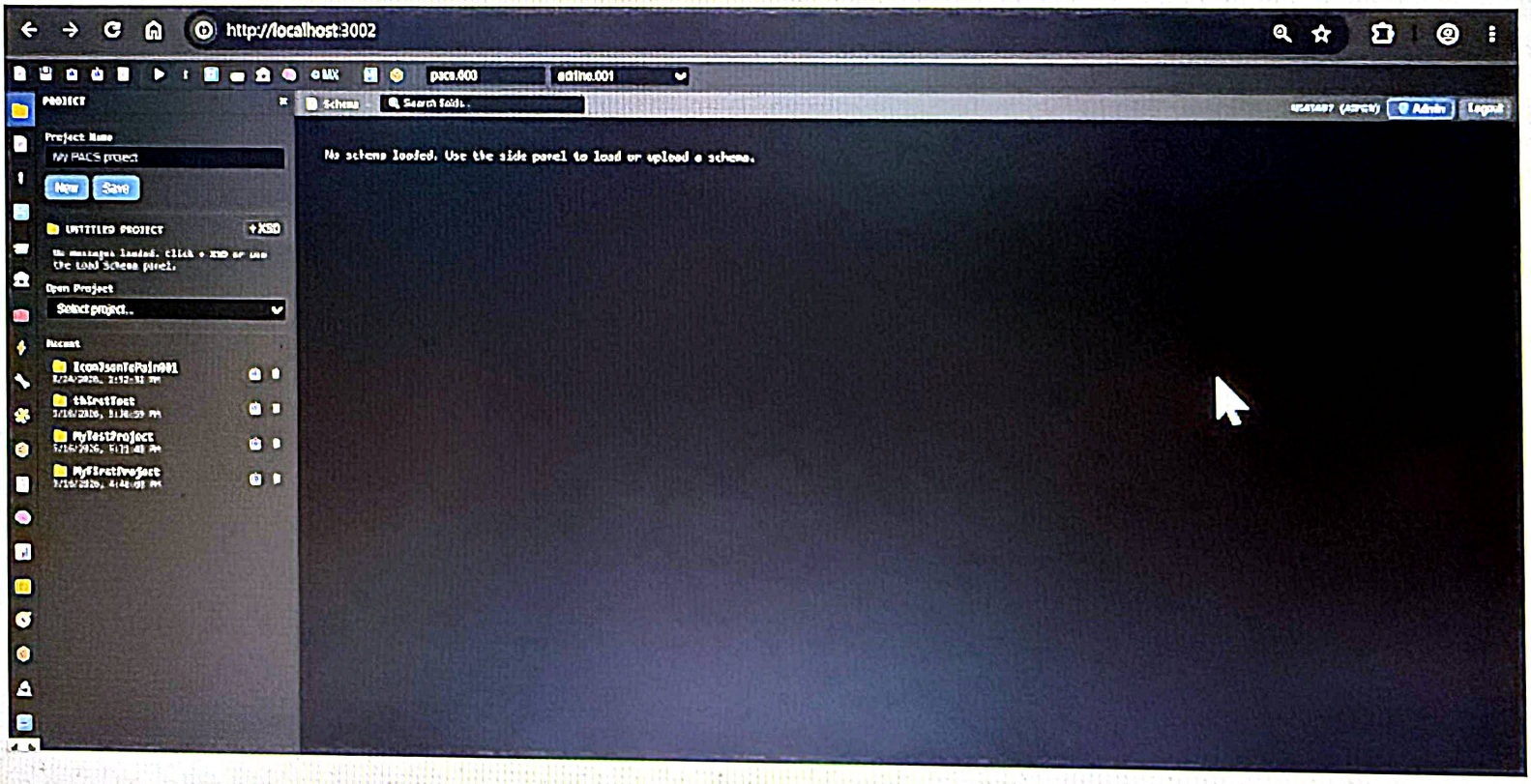
---

17) Engine API and embedding patterns

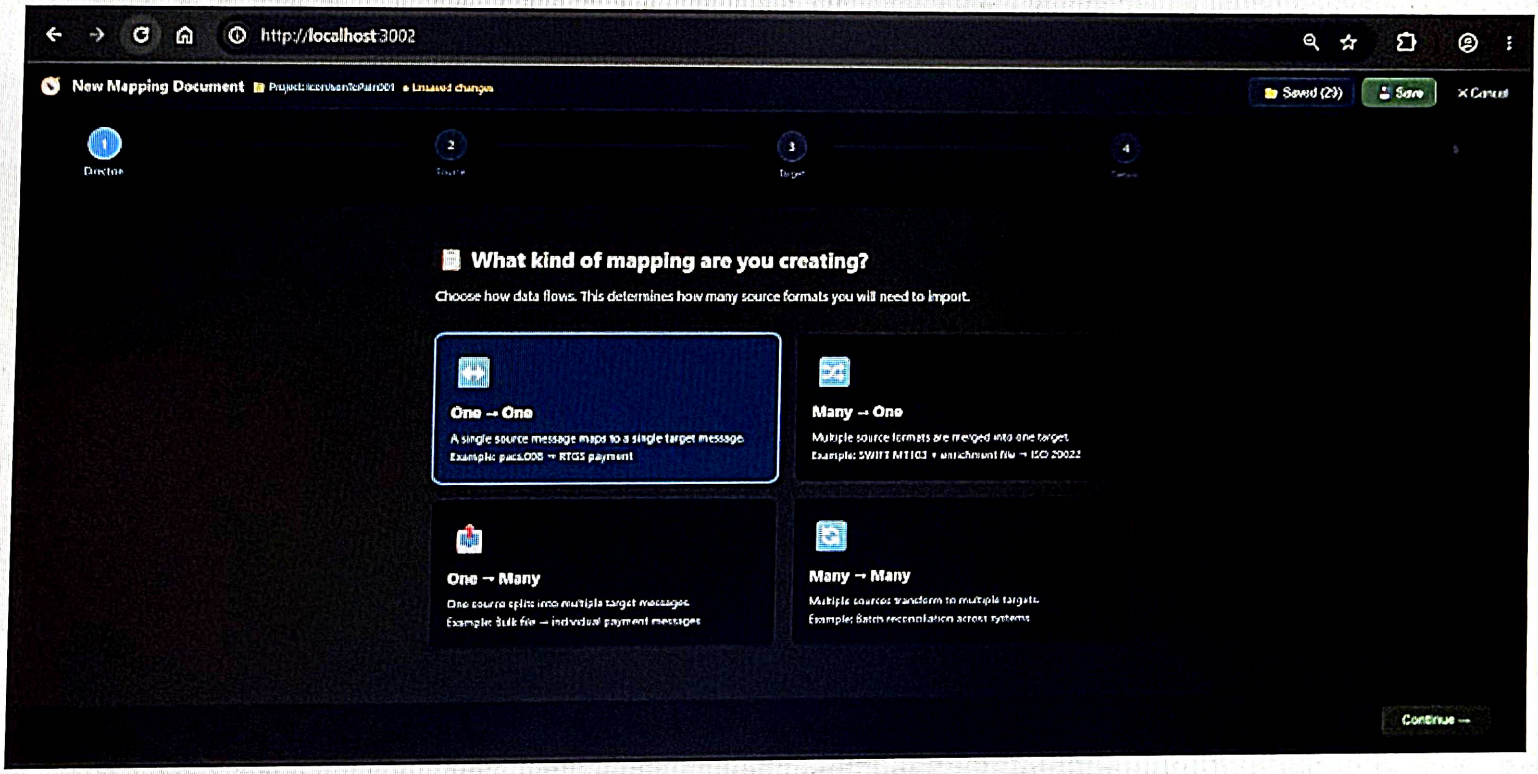
17.1 Core abstractions

- `TransformationEngine`

UI Screenshot 1: img.png



UI Screenshot 2: img_1.png



- `img_1.png`
- `img_2.png`
- `img_3.png`
- `img_4.png`

For each screenshot in this PDF:
- correlate with the nearest section above
- capture component-level behavior
- map to endpoint and service paths during implementation walkthrough

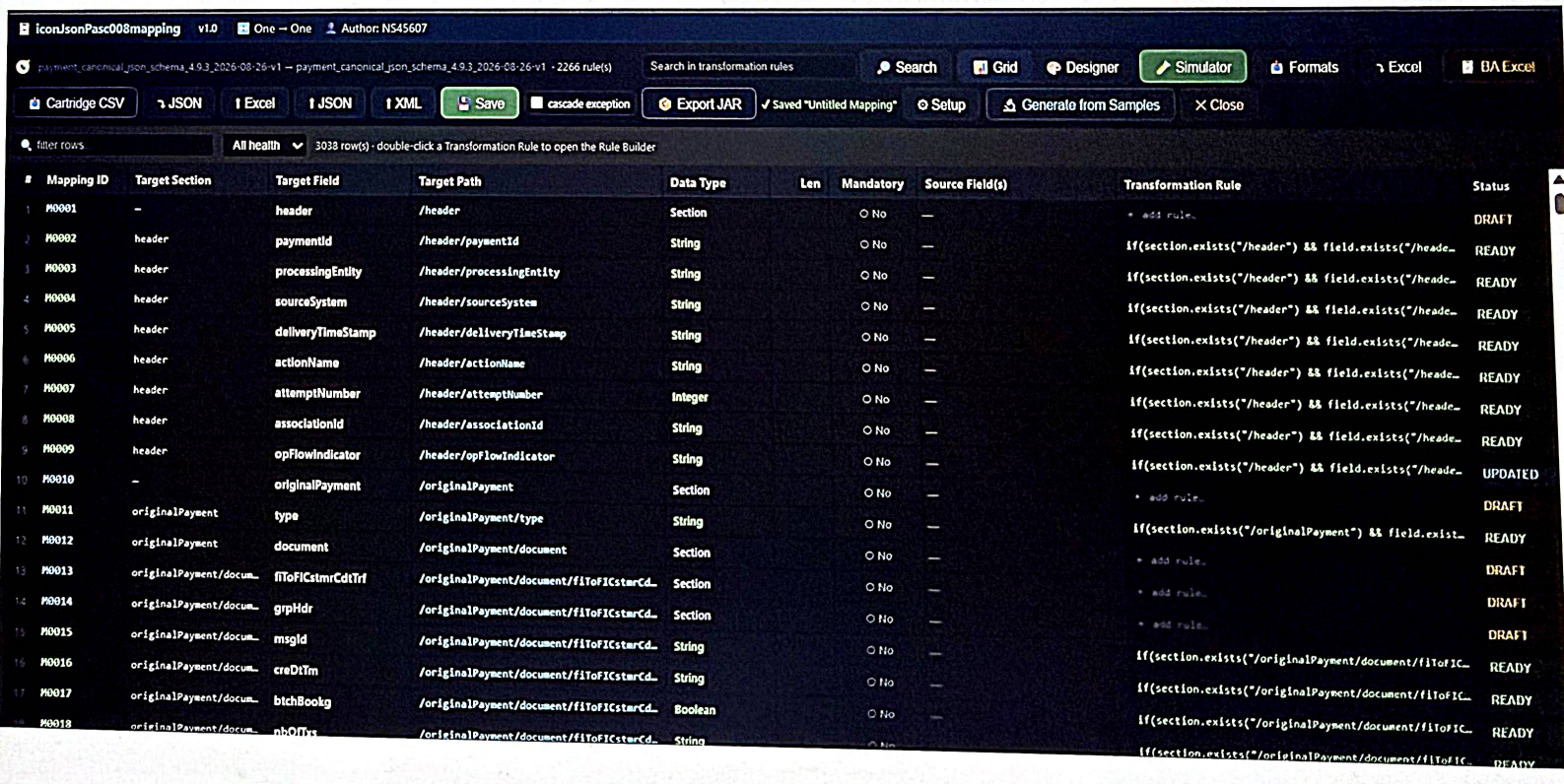
---

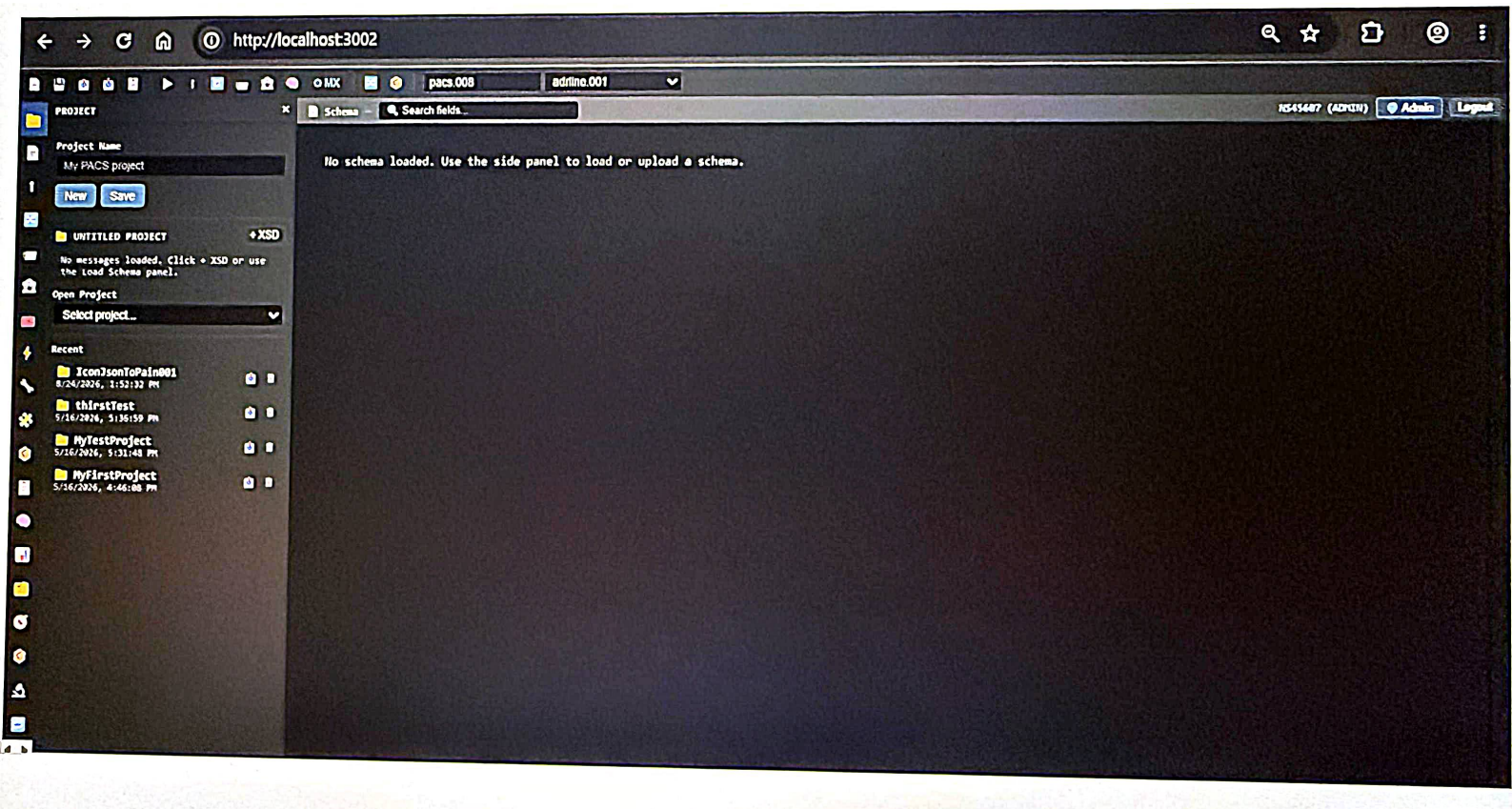
23) Source index used for this handbook

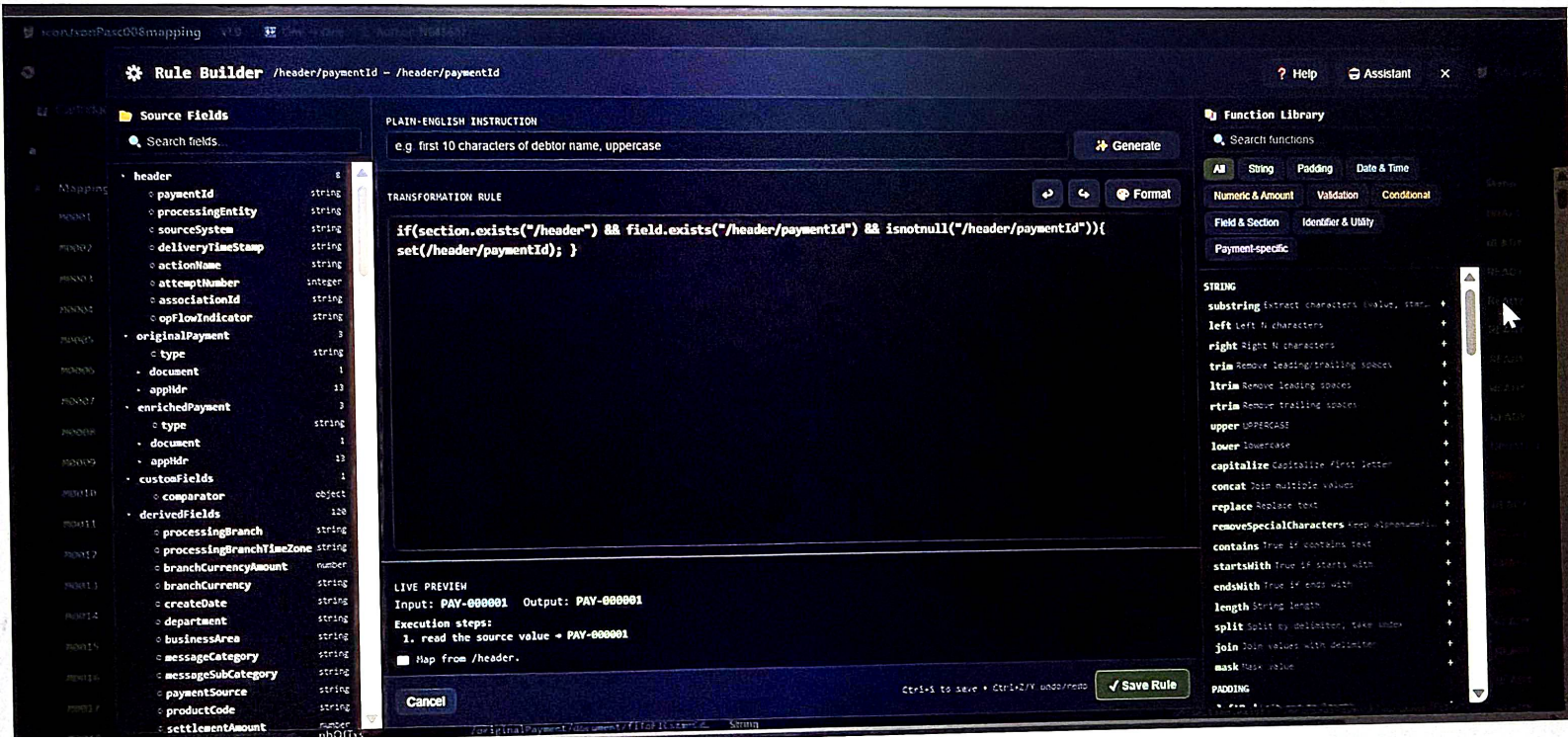
Primary sources:
- `docs/TECHNICAL_DOCUMENT.md`
- `docs/TDD.md`
- `docs/AUTH_MODULE_SPEC.md`
- `docs/TRANSFORMATION_LANGUAGE.md`
- `docs/ENGINE_API.md`
- `docs/MDB_MIGRATION_ROLLBACK.md`
- `docs/MDB_JAR_EXPORT_SPEC.md`
- `docs/MESSAGE_CLASS_MODULE.md`
- `docs/OPTRANSFORMER_JSON_SCHEMA_MULTI_CHILD_COMPOSITION_FIXES.md`
- `docs/02_SAMPLE_SNIPPETS_AND_EXPECTED_OUTPUTS.md`
- `docs/03_PATTERN_INVENTORY.md`
- `docs/06_TEST_ACCEPTANCE_CRITERIA.md`
- `docs/PHASE_4_REPORT.md`
- `docs/PHASE_5_REPORT.md`
- `src/main/resources/application.properties`
- `pom.xml`
- `xsd-ui/package.json`

Supplementary prompt/spec references were used as additive implementation guidance where applicable.

Screenshot of img_2.png




http://localhost:3002
pacs.008
admin.001
PROJECT
Project Name
My PACS project
New
Save
UNTITLED PROJECT
+XSD
No messages loaded. Click + XSD or use the Load Schema panel.
Open Project
Select project...
Recent
IconJsonToPain001
8/24/2026, 1:52:32 PM
thirstTest
5/16/2026, 5:36:59 PM
MyTestProject
5/16/2026, 5:31:48 PM
MyFirstProject
5/16/2026, 4:46:08 PM
Schema
Search fields...
No schema loaded. Use the side panel to load or upload a schema.
Admin
Logout


Rule Builder /header/paymentId → /header/paymentId
Help
Assistant
Source Fields
Search fields...
header 8
paymentId string
processingEntity string
sourceSystem string
deliveryTimeStamp string
actionName string
attemptNumber integer
associationId string
opFlowIndicator string
originalPayment 3
type string
document 1
appHdr 13
enrichedPayment 3
type string
document 1
appHdr 13
customFields 1
comparator object
derivedFields 120
processingBranch string
processingBranchTimeZone string
branchCurrencyAmount number
branchCurrency string
createDate string
department string
businessArea string
messageCategory string
messageSubCategory string
paymentSource string
productCode string
settlementAmount number
PLAIN-ENGLISH INSTRUCTION
e.g. first 10 characters of debtor name, uppercase
Generate
TRANSFORMATION RULE
Format
if(section.exists("/header") && field.exists("/header/paymentId") && isnotnull("/header/paymentId")){
set(/header/paymentId); }
LIVE PREVIEW
Input: PAY-000001 Output: PAY-000001
Execution steps:
1. read the source value → PAY-000001
Map from /header.
Cancel
Ctrl+S to save • Ctrl+Z/Y undo/redo
Save Rule
Function Library
Search functions
All
String
Padding
Date & Time
Numeric & Amount
Validation
Conditional
Field & Section
Identifier & Utility
Payment-specific
STRING
substring
left Left N characters
right Right N characters
trim Remove leading/trailing spaces
ltrim Remove leading spaces
rtrim Remove trailing spaces
upper UPPERCASE
lower lowercase
capitalize Capitalize first letter
concat Join multiple values
replace Replace text
removeSpecialCharacters
contains True if contains text
startsWith True if starts with
endsWith True if ends with
length String length
split Split by delimiter, take index
join Join values with delimiter
mask Mask value
PADDING

- `CompiledMapping`
- `TransformationResult`
- `TransformationDiagnostics`
- `TransformationOptions`
- `TransformationListener`

17.2 Embedding snippet

```java
TransformationEngine engine = new TransformationEngine();
CompiledMapping compiled = engine.compile(mappingDocument);
TransformContext ctx = TransformContext.systemDefault();
TransformationResult result = engine.transform(compiled, sourceMessage, ctx);
String output = result.output();
```

17.3 Explain and diagnostics

- Explain-mode captures reasoning/traces.
- Deterministic audit journal can track hashes, mapping version, and timings.

---

18) Test strategy and acceptance criteria

18.1 Mandatory coverage

Backend:
- Parser tests (XSD/JSON schema and errors)
- Mapping validation and normalization
- Function evaluator edge cases
- Security authorization tests
- Flow simulate/execute tests

Frontend:
- Studio mapping engine parity tests
- Function catalog interaction tests
- API auth/session behavior tests

E2E:
- Upload schema -> build mapping -> test-run -> preview output

18.2 Semantic acceptance from docs

- direct expression interpretation
- OR-list detection and mapping
- section-level extraction behavior
- interpreted columns surfaced in export/UI

18.3 Evidence expectations

- include command output summary
- include changed files list
- include residual risks and rollback notes

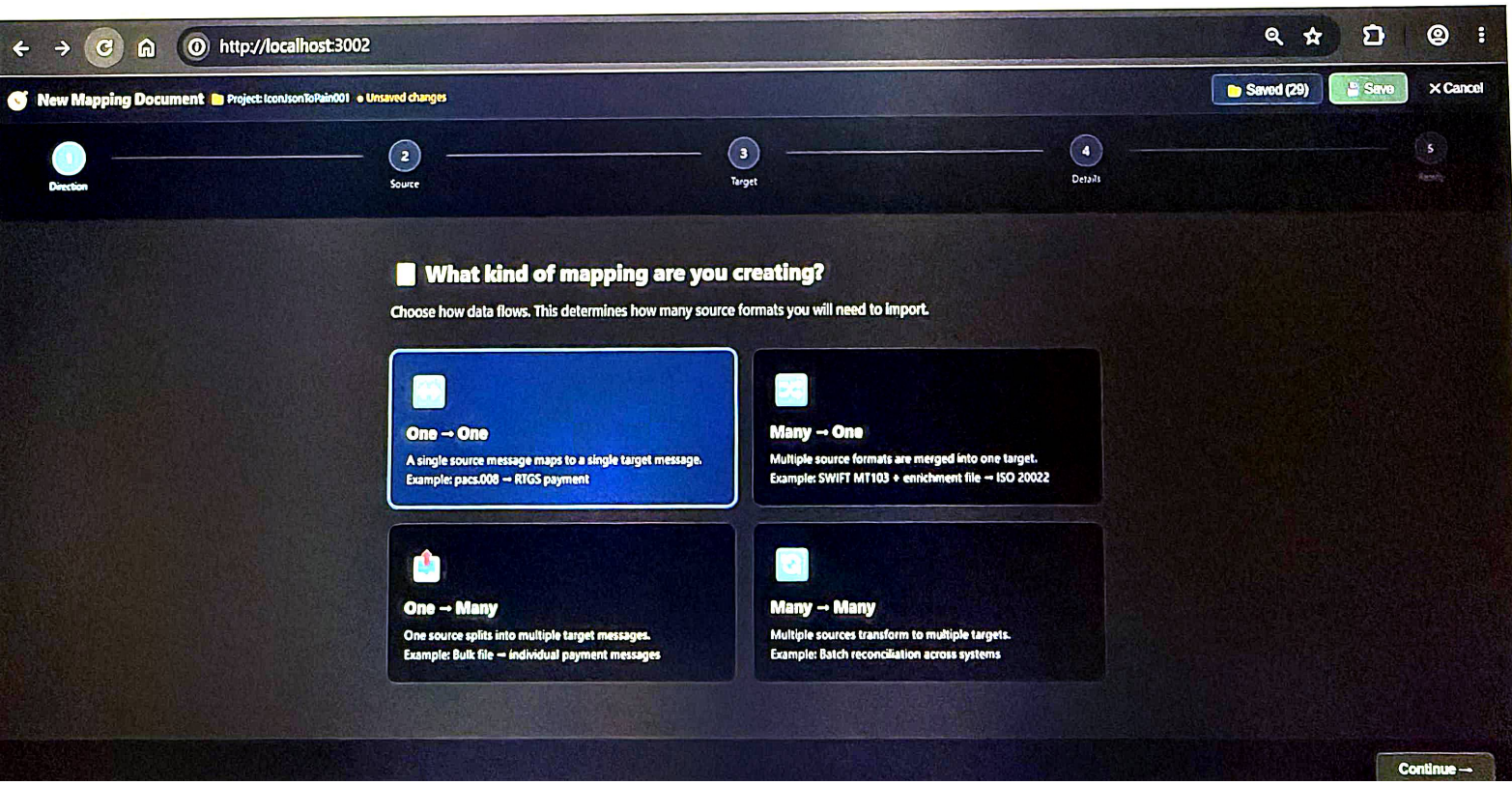
---

19) Phase-by-phase implementation plan (for Claude)

Phase 0: Baseline
- verify build/tests
- inventory toggles and runtime assumptions
- freeze contract baseline

Phase 1: Contract codification
- endpoint matrix + sample payloads
- role constraints and error contracts

Phase 2: Schema onboarding
- robust parser services and version-aware retrieval


http://localhost:3002
New Mapping Document
Project: IconJsonToPain001
Unsaved changes
Saved (29)
Save
Cancel
1
Direction
2
Source
3
Target
4
Details
5
What kind of mapping are you creating?
Choose how data flows. This determines how many source formats you will need to import.
One → One
A single source message maps to a single target message.
Example: pacs.008 → RTGS payment
Many → One
Multiple source formats are merged into one target.
Example: SWIFT MT103 + enrichment file → ISO 20022
One → Many
One source splits into multiple target messages.
Example: Bulk file → individual payment messages
Many → Many
Multiple sources transform to multiple targets.
Example: Batch reconciliation across systems
Continue →